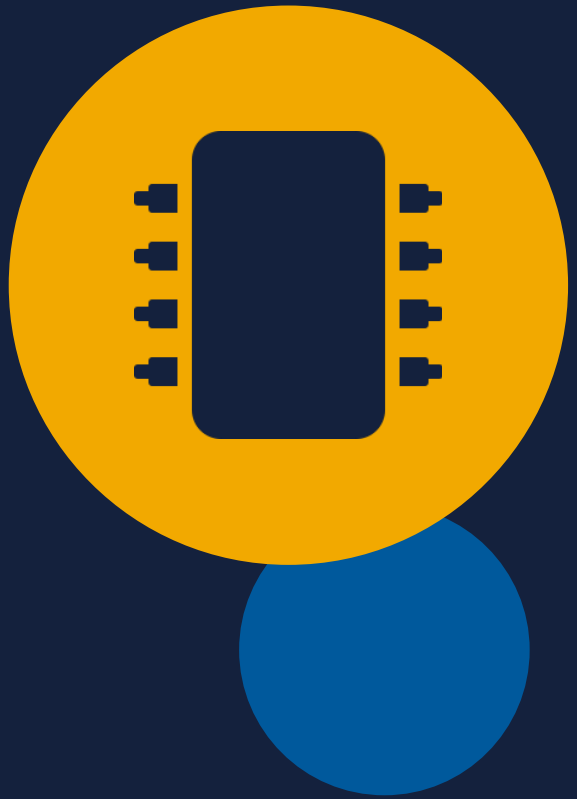
C Programming

A general overview: variables, constants, data types and operators



What is C?

- A general-purpose, procedural programming language
- Compiled to fast native machine code
- Statically typed, with direct memory access through pointers
- The foundation of operating systems and many other languages

1972 Developed by Dennis Ritchie at Bell Labs

C23 The latest ISO standard

Fast Minimal runtime, close to the hardware

Key features



Compiled

Source code becomes native machine code



Statically typed

Every variable has a declared type



Pointers

Direct access to memory addresses



Portable

Runs on almost every platform



Small and fast

Tiny runtime with little overhead



Procedural

Programs are built from functions

C in action

hello.c

```c
#include <stdio.h>

int main(void) {
    printf("Hello, World\n");
    return 0;
}
```

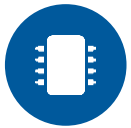
Compile and run

```
$ gcc hello.c -o hello
$ ./hello  ->  Hello, World
```



Operating systems

Linux kernel, Windows internals



Embedded systems

Microcontrollers and firmware



Databases & tools

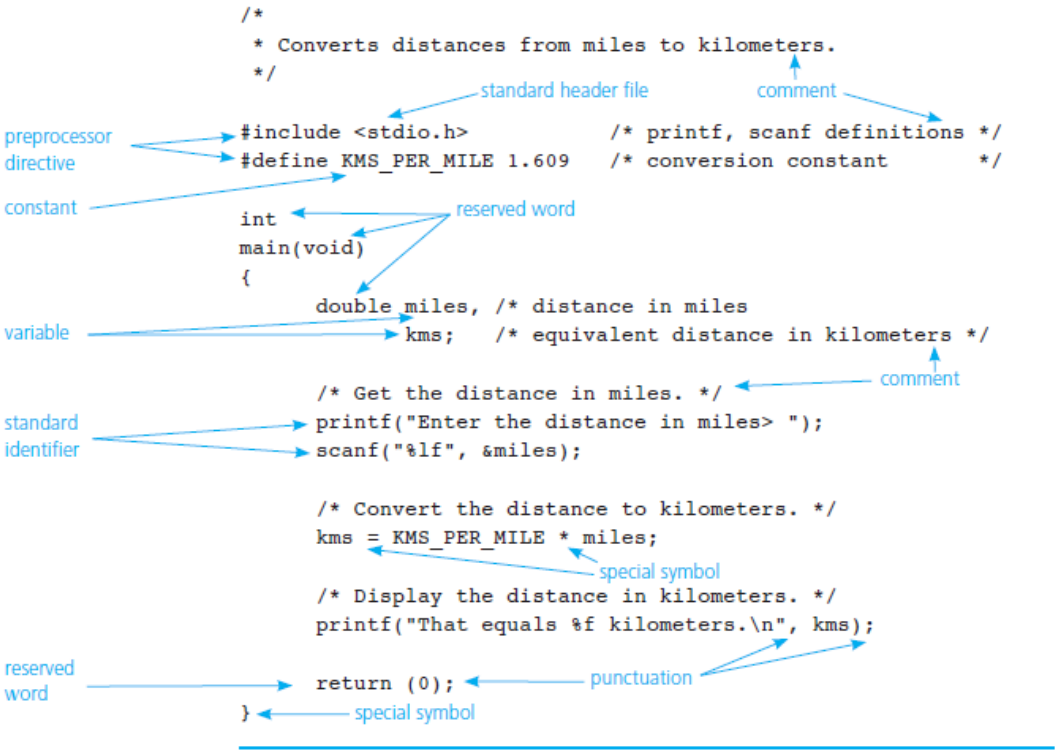
SQLite, compilers, interpreters



Games & drivers

Engines and hardware drivers

General View



- **Preprocessor Directive**
  include, define,…
- **Comments**
  /* …*/, //
- **Variables/Constant**
  int, float, double, …
- **Functions**
  main(), user define function

Variables

variables.c

```c
int age = 25;
float price = 9.99f;
char grade = 'A';
double pi = 3.14159;

int x, y;
x = y = 0;

age = 26;      // new value
// age = "ten"; is an error
```

A variable is a named memory location with a declared type.

Naming rules

- Letters, digits and _ only
- Cannot start with a digit
- Case-sensitive: age ≠ Age
- Not a keyword (int, return)

Constants

constants.c

```c
#define MAX_USERS 100

const double PI = 3.14159;


PI = 3.0;    // compile error
```

`#define`

Preprocessor text substitution, with no type

`const`

A typed value the compiler will not let you change

Basic data types

char

`'A'`

One character

int

`42`

Whole numbers

float

`3.14`

Single precision

double

`3.14`

Double precision

short

`100`

Smaller integer

long

`100000L`

Larger integer

unsigned

`42u`

No negatives

Check sizes with sizeof, e.g. sizeof(int) is typically 4 bytes.

Numbers & chars

Numbers & characters

```
int count = 42;
float f = 2.5;
double d = 2.5233233;
char c = 'A';   // 65
```

int

Typically 4 bytes

double

About 15 digits of precision

Operators at a glance

Arithmetic

`+ - * / %`

Math on numbers

Relational

`== != < > <= >=`

Compare, return 1 or 0

Logical

`&& || !`

Combine conditions

Assignment

`= += -= *= /=`

Store and update values

Bitwise

`& | ^ ~ << >>`

Operate on bits

Special

`++ --`
`?: sizeof`

Increment, ternary, size

Arithmetic operators

Op	Name	Example	Result
+	Addition	`7 + 3`	10
-	Subtraction	`7 - 3`	4
*	Multiplication	`7 * 3`	21
/	Integer division	`7 / 3`	2
/	Float division	`7 / 3.0`	2.333…
%	Remainder	`7 % 3`	1
++	Increment	`x++`	x + 1

Precedence

```
1   ( )
2   ++ --  !
3   * / %
4   + -
```

Highest first. % works on integers only.

Relational & logical

Relational

```
5 == 5        // 1
5 != 3        // 1
5 > 8         // 0
5 <= 5        // 1
```

Logical

```
1 && 0        // 0
1 || 0        // 1
!1            // 0
int age = 20;
age >= 18 && age < 65   // 1
```

Comparisons return 1 (true) or 0 (false). Unlike math, 3 < x < 10 is not a range check.

&& and || short-circuit: they stop once the result is known.

More operators

Assignment

```
int n = 10;
n += 5;     // 15
n *= 2;     // 30
n /= 4;     // 7
```

Increment

```
int i = 5;
i++;        // i is 6
++i;        // i is 7
int j = i--;  // j 7, i 6
```

Ternary & sizeof

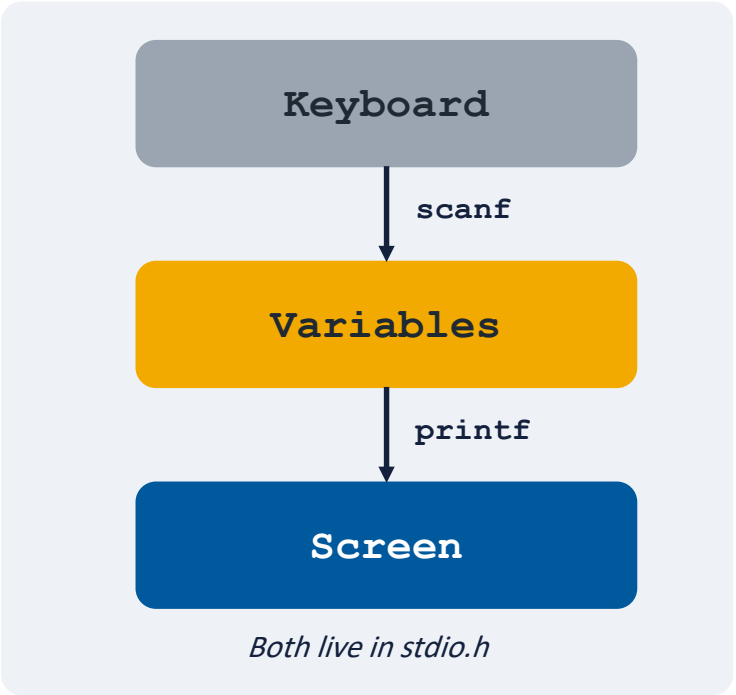
```
int a = 3, b = 9;
int m = (a > b) ? a : b;
// m is 9
sizeof(double)  // 8
```

Bitwise

```
6 & 3   // 2
6 | 3   // 7
6 ^ 3   // 5
1 << 3  // 8
```

Input and output in C

- printf writes formatted text to the screen
- scanf reads formatted values from the keyboard
- Both use a format string with % placeholders
- Both are declared in stdio.h



printf basics

hello.c

```c
#include <stdio.h>

int main(void) {
    printf("Hello, World\n");
    printf("Total: %d items\n", 5);
    return 0;
}
```

Output

```
Hello, World
Total: 5 items
```

Format string

Plain text plus % placeholders

Arguments

One value per placeholder, in order

\n

Starts a new line

Format specifiers

Specifier	Type	Value	Output
`%d`	int	`42`	`42`
`%u`	unsigned int	`42u`	`42`
`%f`	double	`3.14`	`3.140000`
`%c`	char	`'A'`	`A`
`%s`	char * (string)	`"Hi"`	`Hi`
`%x`	unsigned, in hex	`255`	`ff`
`%%`	a literal percent sign	`(none)`	`%`

Width, precision and flags

format.c

```c
printf("[%5d]\n", 42);      // width 5
printf("[%-5d]\n", 42);     // left-align
printf("[%05d]\n", 42);     // zero-pad
printf("[%.2f]\n", 3.14159);
printf("[%8.3f]\n", 3.14159);
printf("[%+d]\n", 42);      // show sign
```

Output

```
[   42]
[42   ]
[00042]
[3.14]
[   3.142]
[+42]
```

Strings, chars and decimals

mixed.c

```c
char name[] = "Amina";
char grade = 'A';
double gpa = 3.82;
printf("%s got %c\n", name, grade);
printf("GPA: %.1f\n", gpa);
printf("%10s|%-10s|\n", name, name);
```

Output

```
Amina got A
GPA: 3.8
     Amina|Amina     |
```

%s

Prints a char array up to its \0

%.1f

One digit after the decimal point

%10s

Right-aligns in a field of width 10

scanf basics

input.c

```c
int age;
double h;

printf("Age and height: ");
scanf("%d %lf", &age, &h);

printf("%d, %.2f\n", age, h);
```

Sample run

```
Age and height: 20 1.75
20, 1.75
```

& operator

scanf needs the address of each variable

%lf for double

printf uses %f, but scanf needs %lf

Whitespace

Spaces and newlines separate numbers

Reading strings and chars

text_input.c

```c
char name[20];
char c;

scanf("%19s", name);    // one word, no &
scanf(" %c", &c);       // skips newline

printf("%s %c\n", name, c);
```

No & for arrays

A char array name is already an address

Width limit

%19s protects a char name[20]

Space before %c

Skips the leftover newline

Common mistakes

Missing &

```
scanf("%d", n);
```

scanf needs an address. Write &n.

Wrong specifier

```
printf("%d", 3.14);
```

A double printed with %d gives garbage. Match the types.

Unbounded %s

```
scanf("%s", name);
```

Long input overflows the array. Use %19s for char name[20].

Leftover newline

```
scanf("%c", &c);
```

It reads the newline left by earlier input. Use " %c".